CPS610-W26, Assignment-4, Labs 7, 8 – Database Systems-II **Due: March19, 2026, 4pm**

(The TAs will evaluate your work and ask questions about progress and final results in the labs in the weeks of Mar9 & Mar16)

Objectives

- Part1: Implement a two-phase locking (2PL) algorithm to handle concurrent transactions on Databases, and to create tables for transactions' Locks and Logs
- Part2: Working practically with Concurrency Control Techniques on Databases

Parts 1&2-Instructions:

In this Assignment, you will simulate a Two-Phase-Locking algorithm to be applied on a Relational Database created by Oracle DBMS, using an object-oriented language preferably Java. In your algorithm:

- A Lock-Table should be created to record all required information for 2PL locking process, and
- A Log-Table should be created to record all required information about committed/uncommitted transactions and the tasks/operations that have been performed in each transaction.

Your algorithm should implement the following functionalities:

a) Locking Protocol [24 Marks]

You are given a schedule of operations/tasks for multiple transactions that will be executed concurrently. Each task can be defined with its relevant format as (Note-each Record is relevant to one data item):

- W (< Record_Id >, < Value > ; Write the Value at the record with Record_Id
- R (< Record_Id >) ; Read the Value from the record with Record_Id
- C ; Commit the relevant Transaction

For example the following shows two schedules for two transactions:

T1: W(1, 5); R(2); W(2, 3); R(1); C

T2: R(1); W(1, 2); C

Your program should implement the 2PL algorithm using both "Shared" (or "Read") and "Exclusive" (or "Write") locks. It should also follow the 2PL rules as discussed through the lectures and described in the class examples. During this algorithm, the Lock-Table which records the lock status of each data item and the Log-Table should be maintained. The tasks/operations in each transaction could be:

Read: In case of a "Read" task, try to make a **Shared Lock** on that record and then Read the value.

A Shared Lock can be acquired only if no Exclusive Lock already made on that record by another transaction.

- If no lock existed on that record from any transactions, a Shared Lock can be acquired on it by the current transaction.
- If the current transaction already made a Shared or Exclusive lock on that record, no lock needs to be acquired or added on it again.
- If another transaction already made an Exclusive lock on that record, then the current transaction should wait until having access to it.

Write: In case of a "Write" task, try to make an **Exclusive Lock** on that record and then Write the value.

An Exclusive Lock can be acquired only if no other lock already made on that record by another transaction.

- If no lock existed on that record from any transactions, an Exclusive Lock can be acquired on it by the current transaction.
- If the current transaction already made an Exclusive Lock on that record, no lock needs to be acquired on it again.
- If the current transaction already had a Shared Lock on that record, that Shared lock should be upgraded to an Exclusive lock.
- If another transaction already made an Exclusive lock on that record, then the current transaction should wait until having access to it.

Commit: In case of a "Commit", release all locks that are made/owned by that transaction.
Wait: In case a lock cannot be acquired, then the transaction should go to wait state and the algorithm should continue with the next transaction according to their assigned priorities. The next time the transaction is processed, this should be checked if the same lock can be acquired for it. The next processing time for each transaction is defined through a "**Round-Robin**" method as follows, Consider:

- The task1 of transaction1, then task1 of transaction2, and so on;
- Then task2 of transaction1, and task2 of transaction2, and so on;
- Then task3 of transaction1, and task3 of transaction2, and so on;

Deliverables for (a):

- A summary about the problem and your suggested techniques/data structures to implement the Concurrent Transactions using 2PL algorithm above on a Database. Also add the source code of your algorithm with proper comments.
- To test your algorithm, try to run the following transactions concurrently using the round-robin method. Display the generated Lock-Table in this process while running the Tasks. Specifically, show the lock status for each Task while the Lock-Table is incrementally created. Also show the order of the Tasks in the final schedule.

  T1: W(1, 5); C
  T2: R(9); R(7); C
  T3: R(1); C

- The following could be a typical order of the tasks in the final schedule. Show the result for your suggested schedule too:
  Round1) → T1:W(1,5); T2:R(9); Round2) → T1:C; T2:R(7); T3:R(1); Round3) → T2:C; T3:C;

b) System Log [11 Marks]

Create and maintain a Log-Table of tasks during the run of the database transactions. This Log Table will be used to roll-back the transactions at any points of crashing, but there is no need to consider any recovery process in this part.
Add entries to the Log-Table, only if the transactions are successfully completed and not at the time that they are seen in the schedule. Therefore, it is recommended to save the entry first in a temporary buffer, and in case of a successful task then it can be saved in the Log-Table. Also, if a task should wait for a lock, then it should not be added to the Log-Table until the task be executed. Make the Log-Table based on the following format:

In case of a **Write** task, store the following information:

- Timestamp; can be a simple counter starting from 0 and incremented by 1 for each log entry
- The Transaction Id
- The Record Id
- The old value stored in that record
- The new value stored in that record
- The timestamp of the previous log entry for this transaction (to be used for Roll-back)

In case of a **Read** task, store the following information:

- Timestamp; can be a simple counter starting from 0 and incremented by 1 for each log entry
- The Transaction Id
- The Record Id
- The value read from that record
- The timestamp of the previous log entry for this transaction (to be used for Roll-back)

In case of a **Commit,** store the following information:

- Timestamp; can be a simple counter starting from 0 and incremented by 1 for each log entry
- The Transaction Id
- The timestamp of the previous log entry for this transaction (to be used for Roll-back)

Deliverables for (b):

- A summary about the problem and your suggested solution regarding the generation of the system Log-Table for concurrent transactions on a database. Also add the source code of your algorithm embedded with proper comments.
- To test your algorithm, try to run the following schedule for concurrent transactions. Display the generated Log-Table in this process while running the Tasks. Specifically, show the log-Table status for each Task while it is incrementally created.

T1:W(1,5); T1:R(2); T1:W(2,3); T1:R(1); T1:C; T2:R(1); T2:W(1,2); T2:C;

W: 0, T1, 1, 1, 5, -1
R: 1, T1, 2, 2, 0
W: 2, T1, 2, 2, 3, 1
R: 3, T1, 1, 5, 2
C: 4, T1, 3
R: 5, T2, 1, 5, -1
W: 6, T2, 1, 5, 2, 5
C: 7, T2, 6

Deliverables & Marking Schema

- Students are required to go through the instructions above, discuss their solution via team work, and finally submit their solution through D2L by the due date. Only ONE submission is required for each team.
- Teams are also required to indicate their partial work to the TA, asking their questions and getting directions from the TA during the lab times until the due.
- The TA will mark assignments by individual assessment (during the labs) for partial and then completed work as well as evaluation of the final submission.
  Lab attendance is mandatory to get the marks for both individual assessment and submitted document evaluations.
- The marks for Labs-7&8 (Assignment-4) will be based on the followings:
  1) Individual assessments in the Lab sessions during the week of Mar9 and Mar16 (only for attended members)
  2) Evaluation of the submission by Mar19, 4pm (only for attended members in the Labs)
  [Total 35 Marks = weight 7%]

Bonus Part-Instructions: **Due: by the due of A5**

This part is optional and only for the purpose of obtaining bonus marks. The objective is implementation of the algorithm as follows:

Deadlock: [3 Bonus Marks]
Your A4 can be extended for detecting Deadlocks through the following algorithm:

- Try to construct a Wait-graph and detect the cycles in the graph
- If a cycle is detected
  - Select a Transaction to abort to resolve the deadlock
  - Roll-back any changes that Transaction has made
  - Release all locks that Transaction has made

Wait-graph:

1- Create a node for each Transaction
2- If T-X must Wait for a lock held by T-Y
   Draw a directed edge from T-X to T-Y
3- Repeat step-2 to create the Wait-graph among all nodes
4- Run the graph search algorithm to detect all the cycles in the Wait-graph
   Use the graph search algorithms

5- If there is a cycle between X and Y, then they are in a Deadlock state
6- Resolve the deadlock state by Aborting the Transaction with the <u>lowest priority</u>
The priority can be determined based on Timestamp or Id of the Transaction (start time of the transaction)
The Transaction with the highest Timestamp/Id (entered the system at a later time) can have the highest priority, OR can have the lowest priority

7- Apply Abort similar to Commit, by entering the Abort task into the Log-Table first
A: 8, T3, 7 : means Abort T-3 at time 8
8- Repeat steps 6-7 to find all Transactions with lowest priority in the deadlocks to be aborted with no conflicts while being entered into the Log-Table
9- Apply Roll-back for all changes the Transactions (aborted ones) have made in the database; This means the Write tasks should be back to their previous state using the Log-Table (previous values should be re-stored again)
10- Remove the Aborted Transactions from the Wait-graph until no deadlocks remain
11- No need to execute the aborted Transaction, just proceed with the next task in the schedule (already prepared by the round-robin method)

Example: Apply your implemented algorithm on the following Transactions and show the results through screen-shots:
T1: R(1); W(2,1); C
T2: R(2); R(3); W(1,2); C
T3: R(1); W(3,3); C

Ordering of the operations (round-robin):
T1:R(1); T2:R(2); T3:R(1); T1:W(2,1); T2:R(3); T3:W(3,3); T1:C;T2:W(1,2); T3:C;T2: C

Deliverables & Marking Schema for bonus part:
- The teams who would like to work on the bonus part, should extend their program of A4 to implement the algorithm above for the dead-locks. Also indicate how their algorithm work on the example provided in the bonus part using the screen-shots. Submit the code and the screen-shots together to gain 3 bonus marks.
- Students are required to go through the instructions above, discuss their solution via team work, and finally submit their solution through D2L by the due date. Only ONE submission is required for each team.
- The bonus part can be submitted by the due of A5.

Good Luck.